

Intent Engineering Frameworks for AI Agents: A Technical Reference

Overview

Intent engineering is the emerging discipline of structuring the objectives, constraints, and decision boundaries that govern AI agent behavior — going beyond prompt engineering to design the full information architecture an agent needs to operate reliably. The core premise across all frameworks: agents fail not because of capability gaps, but because intent is underspecified. This reference covers six frameworks and methodologies that define how practitioners codify intent for autonomous systems.[1][2]

Huryn's 7-Part Intent Structure

Origin and Attribution

Paweł Huryn published "The Intent Engineering Framework for AI Agents" on January 12, 2026 via The Product Compass, his Substack newsletter. Huryn is a product management thought leader whose framework draws on OKR methodology (Christina Wodtke) and empowered team objectives (Marty Cagan). The framework has been widely shared and cited across the AI agent community.[2][3][4]

The Core Thesis

Huryn defines intent as "what determines how an agent acts when instructions run out". Context without intent is noise — an agent can have rich contextual information but still optimize the wrong thing if objectives, outcomes, and constraints are underspecified. The framework deliberately mirrors how product teams set objectives for

empowered human teams, then extends that structure with agent-specific controls.[2]

The Seven Components

The structure splits into two groups. The first four components apply to any delegation — human or AI:

1. Objective (The Problem + Why): Aspirational and qualitative. It defines the problem being solved and why it matters. A weak objective says "Handle customer support tickets." A strong one says "Help customers resolve issues quickly so they can get back to work, without creating more frustration than they started with." The why enables the agent to reason about edge cases and make better autonomous decisions.[2]

2. Desired Outcomes (Observable States): Not activities — observable state changes from the user/stakeholder perspective. Outcomes must be measurable or verifiable without relying on agent self-report, and leading rather than lagging. Two to four outcomes per objective is the target range.[2]

3. Health Metrics (Goodhart Prevention): Non-regression metrics that define what must not degrade while pursuing outcomes. Without them, "resolve issues faster" leads the agent to rush and reduce quality, "reduce escalations" leads it to handle things it shouldn't. Health metrics steer the prompt layer — they inform how conservative the agent should be when making trade-offs. This is distinct from hard guardrails, which block actions entirely.[2]

4. Strategic Context (System Boundaries): The system the agent operates within — what else is happening, what other teams or agents are doing, what the broader objectives are. A 2024 paper (arXiv:2401.04729) demonstrated that supplying strategic context beyond raw task specifications significantly improves AI autonomy. [2]

The next three components extend the structure for agents specifically:

5. Constraints (Steering vs. Hard Enforcement): Two types: steering constraints live in the prompt layer and guide behavior ("prefer brevity"), while hard constraints are enforced architecturally in the orchestration layer, not in prompts ("never access production database directly"). The distinction matters because prompt-layer constraints can be circumvented, while architecture-enforced constraints cannot.[2]

6. Decision Types & Autonomy Boundaries: Explicitly define which decisions the agent may take autonomously versus which require escalation. This maps directly to the autonomy levels concept — different decision types get different levels of agent independence.[4]
[2]

7. Stop Rules (Halt, Escalate, Complete): When the agent should stop working, when it should escalate to a human, and how it knows it's done. Without explicit stop rules, agents may loop indefinitely on unsolvable problems or declare completion prematurely.[2]

Concrete Example

Objective: Help customers resolve Tier-1 issues without frustration

Desired Outcomes:

- Customer confirms their issue is resolved
- No follow-up ticket on same topic within 24 hours
- Customer rates interaction as helpful

Health Metrics:

- CSAT must stay above 4.2
- Repeat contact rate must not increase
- Escalation quality score must stay stable

Constraints (Steering): Prefer resolution over speed

Constraints (Hard): Cannot issue refunds > \$50, cannot access billing system

Decision Types:

- Autonomous: FAQ responses, status checks, basic troubleshooting
- Escalate: Billing disputes, account security, emotional distress

Stop Rules:

- Halt: If 3 consecutive attempts fail to resolve
- Escalate: If customer expresses frustration twice
- Complete: Customer confirms resolution or 24h pass with no follow-up

The Pathmode IntentSpec Format

Origin and Attribution

IntentSpec is an open standard for spec-driven development created by Pathmode (pathmode.io), with Lewis Hall identified as a key figure at the company. The standard is published at intentspec.org with a formal JSON Schema, a GitHub Action for CI/CD validation, and integrations with major AI coding agents.[5][6]

The Five-Part Structure

IntentSpec is a structured document format designed to be committed alongside code in a repository as an intent.md file:[7][5]

- 1. Objective:** What is being built and why. States the problem, the user segment, and the rationale. Gives the agent enough context to make judgment calls when the spec doesn't cover a scenario.[7]
- 2. User Goal:** What the user is trying to accomplish — the job-to-be-done, written from the user's perspective. Prevents scope creep by anchoring every decision to a specific user outcome.[7]
- 3. Outcomes:** What success looks like. Measurable criteria that can be verified programmatically or through testing. Makes "done" unambiguous.[7]
- 4. Edge Cases:** What could go wrong. Boundary conditions, error states, and scenarios the implementation must handle. Prevents the agent from making naive happy-path-only assumptions.[7]
- 5. Verification:** How to confirm it's done. Specific test cases, acceptance criteria, or automated checks that close the feedback loop.
[7]

Spec-Driven Development Model

IntentSpec positions itself as the antidote to "vibe coding" — the practice of prompting an AI agent conversationally and hoping the output is correct. The core argument: prompts are ephemeral and don't survive context switching, while an IntentSpec is a structured contract that lives in the repo and tells any agent what "Done" looks like before it writes a line of code.[5]

The workflow follows three steps: write an intent.md → point the AI agent to it (via [CLAUDE.md](#), .cursorrules, or MCP context) → validate in CI using a GitHub Action that enforces schema compliance on every PR.[5]

Concrete Example

```
---
id: "INT-EXPORT-JSON-001"
objective: "Allow users to export filtered data to JSON"
outcomes:
  - "User clicks Export → JSON file downloads to device"
  - "Export contains only the currently filtered dataset"
  - "No PII fields are included in the output"
constraints:
  - "Must run client-side only (no server round-trip)"
  - "File size must not exceed 10MB"
edgeCases:
  - scenario: "Empty dataset"
    expected: "Show toast: 'Nothing to export' — no file created"
  - scenario: "10k+ rows"
    expected: "Stream to file, show progress indicator"
healthMetrics:
  - "Page load time must not increase"
  - "Existing CSV export must continue to work"
---
```

IntentSpec vs. AGENTS.md

Feature	AGENTS.md	IntentSpec
Format	Unstructured text	Markdown + YAML + JSON Schema
Validation	None (vibe check)	Automated (CI/CD enforceable)
Goal	Context injection	Application logic standard
Testability	Low	High (deterministic)

5

Action Schemas

What They Are

An agent action schema is a formal specification describing the actions an intelligent agent can perform within a system. It establishes structured parameters for agent behavior, including input requirements, execution processes, expected outcomes, and error handling protocols.[8]

Core Components

Every action schema contains five fundamental elements:[8]

- **Action Identity:** Unique identifiers, names, versions, and taxonomical classifications
- **Input Parameters:** Defined data types, validation rules, required vs. optional fields
- **Execution Logic:** Step-by-step procedures for processing inputs and generating outputs
- **Output Specifications:** Expected return formats, data structures, and success indicators
- **Error Handling:** Predefined responses to failure scenarios, retry mechanisms, and fallback procedures

Most modern implementations use JSON-LD formatting to enable semantic understanding and interoperability, and often integrate with OpenAPI specifications.[8]

How Action Schemas Differ from Tool Whitelists

A tool whitelist is a binary access control mechanism — it determines whether an agent *can* call a particular tool. An action schema goes further by defining *how* the tool should be called, *what inputs are valid*, *what outputs to expect*, and *what to do when things fail*. Security practitioners recommend allowlists for the tools an AI agent can access — anything not on the list is off-limits by default — but the schema provides the semantic meaning and execution context that a simple list cannot.[9][8]

Claude Code's Permission System

Claude Code implements action control through a tiered permission system with allow, ask, and deny rules that can be configured at user, project, or managed levels:[10]

Tool Type	Example	Approval Required
Read-only	File reads, grep	No
Bash commands	Shell execution	Yes
File modification	Edit/write files	Yes

Permissions can be set via the `/permissions` command, CLI flags (`--allowedTools`, `--disallowedTools`), or `settings.json` configuration files. The deny rules always take precedence, followed by ask, then allow. Permission modes range from default (prompts on first use) to `bypassPermissions` (skips all prompts), with `dontAsk` mode auto-denying tools unless pre-approved — a pattern that enforces action schemas at the configuration level rather than the prompt level.[11]
[10]

The `--disallowedTools` flag removes specific tools from the model's context entirely, which is functionally stronger than a deny rule — the agent doesn't even know the tool exists. This maps to the action schema concept of constraining the agent's action space at the

architecture layer rather than relying on prompt-level instructions.
[12]

Autonomy Level Framework

Origin and Attribution

Published in June 2025 by K. J. Kevin Feng, David W. McDonald, and Amy X. Zhang at the University of Washington (arXiv:2506.12469). The paper appeared in the Knight First Amendment Institute's "AI and Democratic Freedoms" essay series. The framework argues that autonomy should be a deliberate design decision, separate from an agent's capability and operational environment.[13][14]

The Five Levels

The framework is user-centered — each level is defined by the role the user takes when interacting with the agent:[15][13]

L e v el	Use r Rol e	Must-Have Controls	Agent Behavior	Example Systems
L 1	Ope rat or	User- managed planning; invocation before actions	Agent requires explicit invocation; avoids preference- based decisions	ChatGPT Canvas, Microsoft Copilot
L 2	Coll abo rat or	Control transfer; shared progress representati on	User and agent plan/delegate/execut e together; user can take over anytime	OpenAI Operator
L 3	Con sult ant	Rich feedback elicitation interfaces	Agent plans and executes most tasks; consults user for expertise and preferences	Gemini Deep Research, GitHub Copilot Agent
L 4	App rov er	Approval elicitation for consequenti al actions	Agent operates independently; seeks user only at failure states or high-risk moments	Devin, Manus
L 5	Obs erv er	Emergency off switch	Fully autonomous; user can only monitor via activity logs	Voyager

Key Design Implications

The framework makes an important distinction: a capable agent (one that performs well on benchmarks) can still be designed to operate at a low autonomy level. Conversely, a less capable agent can operate at higher autonomy when tasks are well-scoped. This decoupling means autonomy is a design choice about user experience and risk tolerance, not just a reflection of model quality.[13]

At L1, the user drives all long-term planning and the agent only provides on-demand assistance — ideal for high-stakes workflows where accountability matters. At L2, both parties plan and delegate, with fluid control handoffs. At L3, the agent takes initiative but actively consults the user for directional guidance. At L4, the user's role becomes passive — only approving at specified gates. At L5, the only user control is an emergency off-switch.[13]

The authors also propose "autonomy certificates" — digital documents issued by a third-party governing body that prescribe the maximum autonomy level an agent can operate at, given its technical specifications and operational environment.[13]

Mapping to Huryn's Framework

The autonomy levels map directly to Huryn's "Decision Types & Autonomy Boundaries" component. An L1 agent would have nearly all decisions requiring escalation, while an L5 agent would have nearly all decisions marked as autonomous. The stop rules in Huryn's framework correspond to the "must-have controls" at each autonomy level — particularly the emergency off-switch at L5 and the approval gates at L4.[13][2]

The Autonomous Preamble Pattern

What It Is

The "autonomous preamble" is a practitioner pattern for framing system prompts when agents operate without human presence — in headless, background, or fully unattended execution modes. This is not a single published framework but rather an emerging set of

conventions observed across Claude Code's SDK, Augment Code's prompting guides, and the broader agentic AI community.[16][17]

The Interactive → Autonomous Shift

When agents shift from interactive (human-in-the-loop) to autonomous execution, several fundamental changes occur in how system prompts are structured:[18][16]

Dimension	Interactive Mode	Autonomous Mode
Permission model	Ask before acting	Act, then verify
Planning	User-guided or collaborative	Self-directed with constraints
Error handling	Pause and ask user	Retry, adapt, or escalate to logs
Scope boundaries	Implicit (user corrects drift)	Explicit (must be codified)
Stop conditions	User says "done"	Programmatic completion criteria
Context refresh	User provides missing info	Agent must gather its own context

Key Components of an Autonomous Preamble

Based on analysis of production system prompts and practitioner guides, autonomous preambles typically include:[19][20][16]

- **Identity and scope:** "You are an [AGENT ROLE] whose goal is to [HIGH-LEVEL OBJECTIVE]" — establishing the agent's role and operational domain
- **Available tools and resources:** Explicit enumeration of what the agent can access, including files, APIs, and execution environments
- **Decision-making strategy:** How to handle ambiguity, which decisions to make autonomously, when to log rather than ask

- **Constraints and prohibitions:** What the agent must never do, especially important when no human is watching
- **Completion criteria:** How the agent knows it's done — verification checks, test suites, or output validation
- **Failure modes:** What to do when stuck — retry limits, alternative approaches, graceful degradation
- **Escalation protocol:** Even in autonomous mode, defining when to halt and signal for human review

Claude Code's Implementation

Claude Code enables autonomous execution through `--dangerously-skip-permissions` (equivalent to `--permission-mode bypassPermissions`), which auto-approves all tool calls without human confirmation. The Claude Agent SDK supports custom system prompts via the `systemPrompt` parameter — either appending to Claude Code's built-in prompt or replacing it entirely.[21][22][23]

Background execution is supported through the Task tool, which spawns subagent instances that work autonomously and return results to the main session. Importantly, subagents inherit permission modes — when bypass mode is enabled, all subagents also get full autonomous access.[24][22]

The recommended pattern for safe autonomous execution is containerized environments where the agent cannot access host-level credentials, SSH keys, or browser cookies. This architectural constraint is more reliable than any prompt-level instruction.[22]

The 7-Component Framework Mapping

Origin and Attribution

Published on whatisaframework.com in February 2026, this mapping connects intent engineering vocabulary to an existing "framework methodology" that uses seven structural components to define agent behavior. The site argues that intent engineering, as named by Huryn and others, is the same underlying architecture that framework methodology has used since 2024, with different vocabulary.[1]

The Seven Components

The framework identifies seven structural areas that any agent deployment must address:[1]

1. The agent's **role** and operating identity
2. The **objective hierarchy** — what the agent is optimizing for
3. **Constraints** that apply in the current context
4. **Decision-making** logic at ambiguous branch points
5. **Output standards** — what quality and format are expected
6. **State and context** the agent should maintain
7. **Edge case handling** — what happens at system boundaries

The mapping to intent engineering is direct: every component Huryn describes has a structural equivalent in this methodology. The implementation language differs but the underlying architecture does not.[1]

Practical Significance

The key insight both approaches share: agents don't fail on capability, they fail on context. When an AI agent makes a mistake, it's rarely because it lacks ability — it's because structured context about objectives, constraints, and decision boundaries was missing. The naming shift from "prompt engineering" to "intent engineering" correctly repositions the discipline as systems design rather than input crafting.[1]

Cross-Framework Comparison

Dimension	Huryn (7-Part)	IntentSpec (Pathmode)	Action Schemas	Autonomy Levels	Autonomous Preamble
Primary focus	Agent behavior design	Code implementation specs	Tool/action control	User-agent interaction	Headless execution
Audience	Product managers, AI architects	Software engineers	Platform engineers	Agent designers, researchers	Practitioners deploying agents
Format	Structured documentation	YAML/Markdown in repo	JSON-LD / OpenAPI	Conceptual framework	System prompt patterns
Validation	Checklist-based	CI/CD enforceable[5]	Schema validation[8]	Autonomy certificates[13]	Runtime behavior
Scope	End-to-end agent intent	Feature-level specs	Individual actions	System-level autonomy	Execution mode config
Constraints	Steering + hard enforcement[2]	Constraints field in spec[5]	Input validation + ACL[8]	Controls per level[13]	Prompt + architecture
Health metrics	Explicit component[2]	health Metrics field[5]	Not addressed	Not addressed	Not addressed
Stop cond	Explicit stop	Verification	Error handling	Emergency off-	Completion

Dimension	Huryn (7-Part)	IntentSpec (Pathmode)	Action Schemas	Autonomy Levels	Autonomous Preamble
itions	rules[2]	section[7]	g[8]	switch (L5)[13]	criteria

How They Complement Each Other

These frameworks operate at different levels of abstraction and address different moments in the agent design lifecycle. Huryn's 7-part structure defines *what* the agent should pursue and *how it should reason about trade-offs* — it's the strategic layer. IntentSpec translates that intent into *feature-level implementation contracts* that AI coding agents can execute against — it's the tactical layer. Action schemas define *what the agent is allowed to do* at the tool level — they're the enforcement layer. The autonomy levels framework determines *how much human involvement* the agent design should require — it's the governance layer. And the autonomous preamble pattern configures *how the agent behaves when no human is present* — it's the runtime layer.[8][7][13][2]

A well-architected agent system would use all five: Huryn's structure to define strategic intent, IntentSpec to translate that into implementable specs, action schemas to constrain the agent's tool access, an autonomy level to calibrate human involvement, and autonomous preamble patterns when the agent runs headless.

Important Caveat: The I-5 Framework

The "I-5 Framework" described as a 5-stage methodology (Intent → Interpretation → Information → Instruction → Implementation) does not appear as a widely published, attributable framework in the intent engineering literature. While AWS has published a "5-I Framework" for AI-powered software development (Investigate, Integrate, Interact, Iterate, Impact), this addresses AI integration into the SDLC rather than intent engineering for autonomous agents. Several 5-stage frameworks exist for AI maturity and implementation, but none match the specific "I-5" nomenclature and

stage sequence described. This may be an emerging or informally circulated framework that has not yet been formally published.[25][26][27]

References

1. [Intent Engineering for AI Agents: The Framework Methodology ...](#) - In January 2026, Product Compass published a detailed breakdown of "intent engineering" — the discipl...
2. [The Intent Engineering Framework for AI Agents](#) - The Intent Engineering Framework shows how to structure objectives, outcomes, health metrics, constr...
3. [Intent Engineering Framework for AI Agents - LinkedIn](#) - Finished reading this post from Paweł Huryn. It's easily the best framework I have seen for defining...
4. [Paweł Huryn's Post - LinkedIn](#) - ... autonomy boundaries, stop rules. Agents don't fail because they can ... My full intent engineeri...
5. [IntentSpec | The Open Standard for Spec-Driven Development](#) - IntentSpec: The Open Standard for. Spec-Driven Development. Functional Guardrails for your AI Agents...
6. [Balancing Legacy Systems with AI Innovation | Lewis Hall posted on ...](#) - At Pathmode, we call this the IntentSpec. It's a structured contract—constraints, outcomes, verifica...
7. [IntentSpec — Definition & Guide | Pathmode Glossary](#) - The IntentSpec is a five-part structured specification format that captures product intent precisely...
8. [Agent Action Schema - Adopt AI](#) - Agent action schema defines the structured blueprint that enables AI agents to understand, execute, ...
9. [Securing AI Agentic Workflows: 🛡️ "Tools" as Guardrails for Safer AI](#) - This is akin to a whitelist or allowlist of permitted actions. Security practitioners recommend usin...
10. [Configure permissions - Claude Code Docs](#) - You can view and manage Claude Code's tool permissions with /permissions . This UI lists all permiss...
11. [How to use Allowed Tools in Claude Code - Instructa.ai](#) - Store your Tools Permissions in Claudes settings.json file: User settings are defined in ~/.claude/s...

12. [allowedToolsCLI flags topermissions` settings · Issue #18973 ...](#) - Key: allow. Description: Array of permission rules to allow tool use. Corresponds to the --allowedTo...
13. [Levels of Autonomy for AI Agents Working Paper - arXiv](#) - We define five levels of escalating agent autonomy, characterized by the roles a user can take when ...
14. [\[2506.12469\] Levels of Autonomy for AI Agents - arXiv](#) - In this work, we define five levels of escalating agent autonomy, characterized by the roles a user ...
15. [LEVELS OF AUTONOMY FOR AI AGENTS](#) - This paper introduces five levels of autonomy based on user roles: operator, collaborator, consultan...
16. [How to build your agent: 11 prompting techniques for better AI agents](#) - This post distills field-tested tactics we use at Augment Code to build autonomous agents that behav...
17. [Building Agents with Claude Code's SDK - PromptLayer Blog](#) - Run Claude Code in headless mode. Use it to build agents that can grep, edit files, and run tests. T...
18. [Prompt Engineering for Production AI Agents](#) - Prompt engineering transforms AI from a basic chatbot into an autonomous agent capable of complex, m...
19. [Designing Agentic Prompts: The First Step !](#) - Agentic AI systems don't just need better prompts. They need structured goals, adaptive thinking, an...
20. [System Prompts and Configuration - The Agentic ...](#)
21. [Using Claude --dangerously-skip-permissions Safely - Promptaa](#) - When you run a command with the claude --dangerously-skip-permissions flag, you're giving the AI ful...
22. [Claude Code dangerously-skip-permissions: Why It's Tempting, Why ...](#) - Claude Code dangerously-skip-permissions: Why It's Tempting, Why It's Dangerous · What --dangerously...
23. [Modifying system prompts - Claude API Docs](#) - Learn how to customize Claude's behavior by modifying system prompts using three approaches - output...
24. [The Task Tool: Claude Code's Agent Orchestration System - Dev.to](#) - Table of Contents Introduction What is the Task Tool? How It Works Available Agent...
25. [Glasswing's Five-Stage Agent Autonomy Framework](#) - Glasswing's proprietary AI Agent framework maps the five-stage

evolution of AI agents and their pote...

26. [5-I framework for an AI-powered software development experience](#) - The 5-I framework is built around five key dimensions: Investigate, Integrate, Interact, Iterate, an...
27. [Bridging the Gaps in AI Transformation: An Evidence-Based ...](#) - The Five-Stage Framework for AI Transformation. Scaling AI requires more than deploying algorithms. ...